

The Blueprint of Lore: Digitizing Middle-earth

A visual guide to Semantic Web
Ontologies, Protégé, and SPARQL.

Mario Lezoche



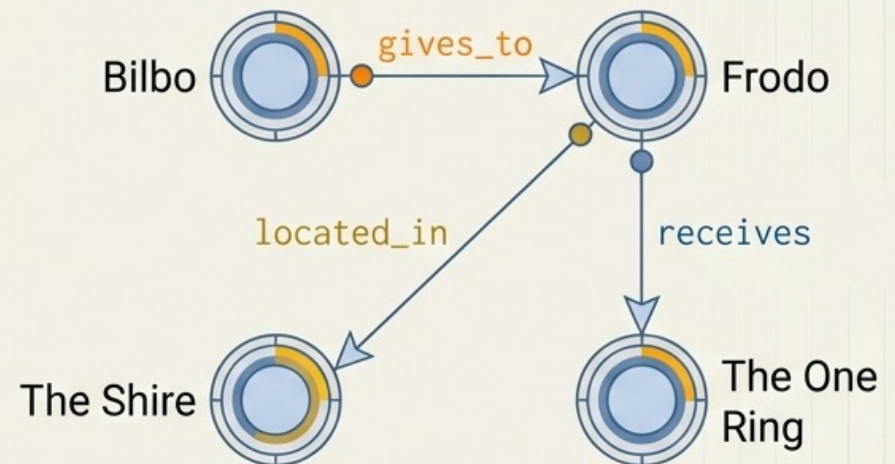
Unstructured Lore vs. Machine Logic

The Lore

Frodo Baggins of the Shire received the One Ring from his uncle, Bilbo. He was tasked with carrying it to Mount Doom.

Humans process narrative. We understand implicit context, history, and meaning.

The Logic



Machines process relationships. They require an explicit, standardized map of concepts to "understand" a domain.

What exactly is an Ontology?

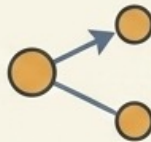
"A formal, explicit specification of a shared conceptualization."
(Gruber 1993; Studer et al. 1998)



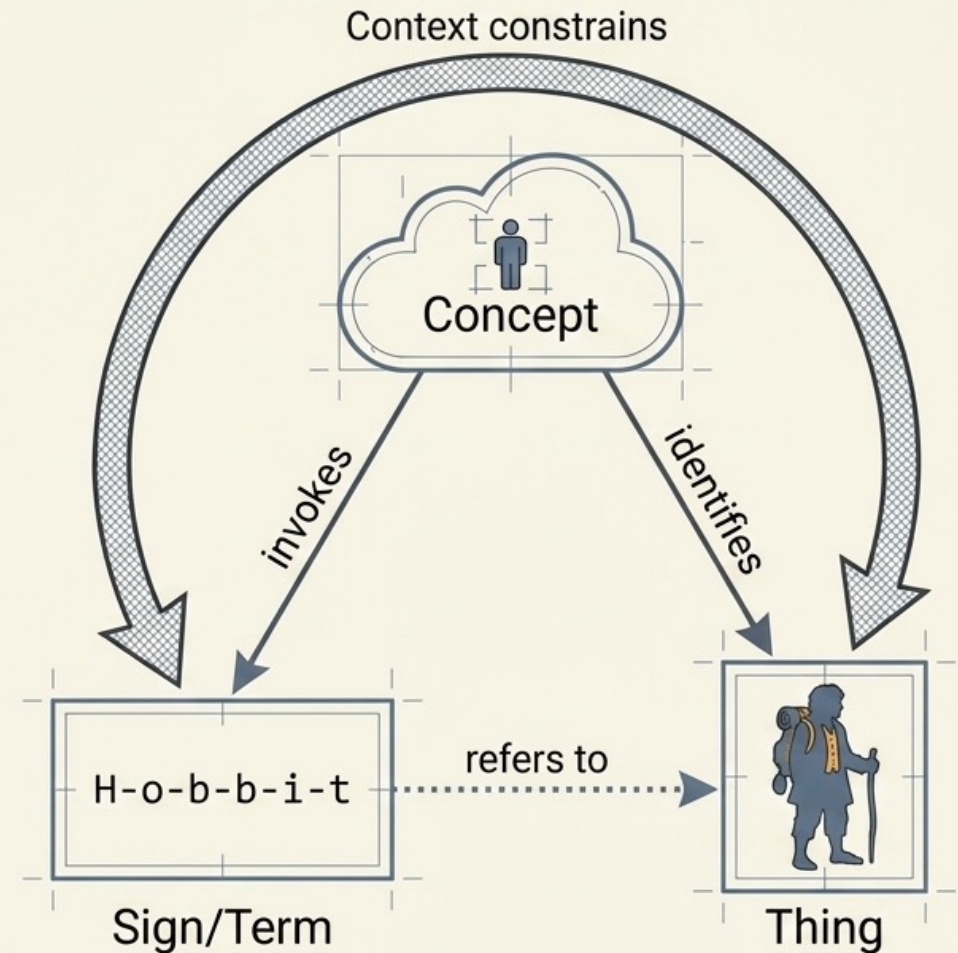
Formal:
Machine-readable
(not just human
prose).



Explicit:
All assumptions
and rules are
defined.

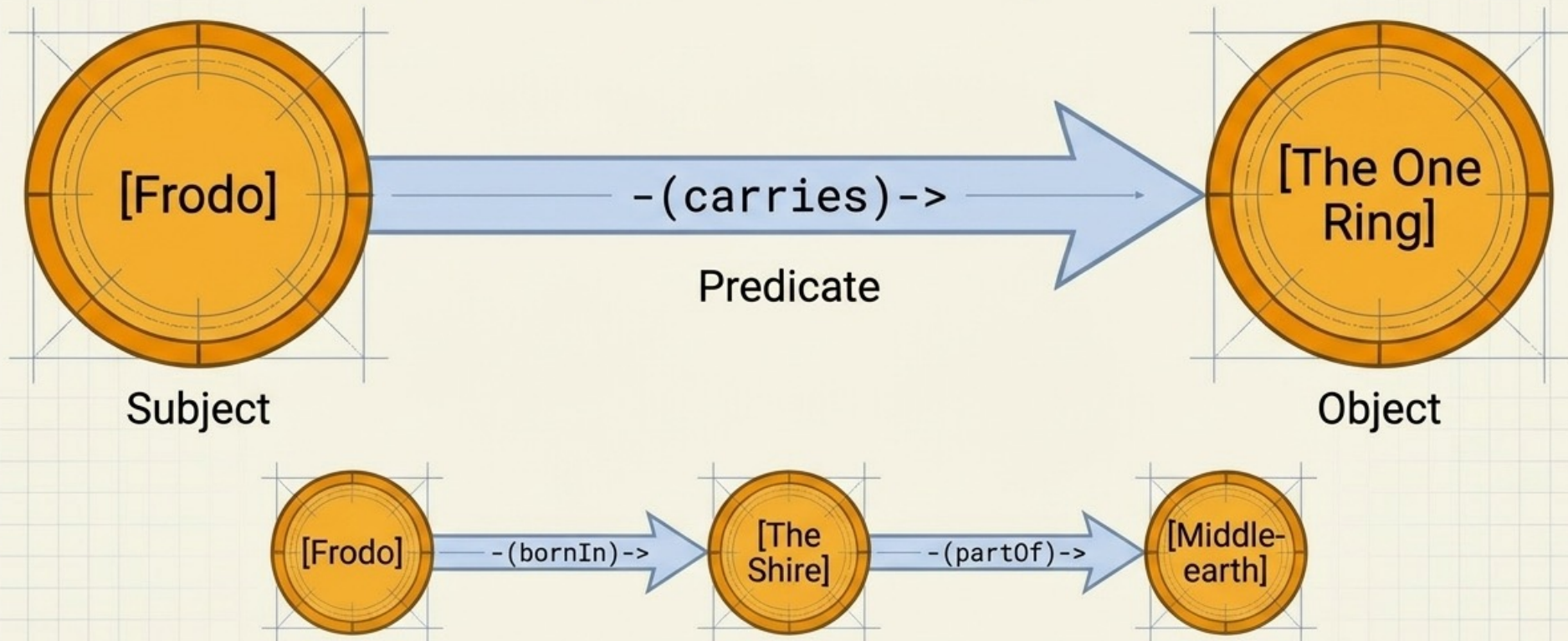


Shared:
A consensual
agreement on
what terms mean.

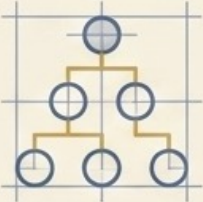


The Grammar of Knowledge: RDF Triples

Every piece of data in the Semantic Web is broken down into simple **three-part** statements. A graph is simply thousands of these triples connected together.



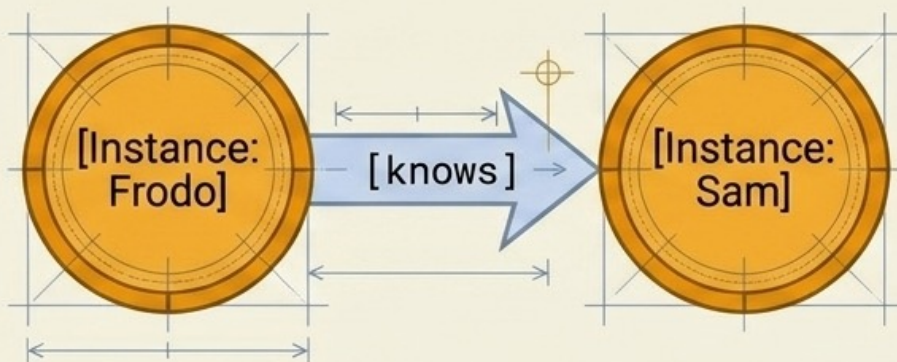
The Building Blocks of a Knowledge Base

Classes	Properties	Instances (Individuals)
		
Sets or categories of things. They form the taxonomy (hierarchy) of your domain.	The relationships that connect different entities together.	The specific, unique objects or people that belong to the classes.
"Hobbit" "Wizard" "Artifact"	"hasRing" "bornIn" "knows"	"Frodo" "Gandalf" "Sting"

Two Types of Relationships

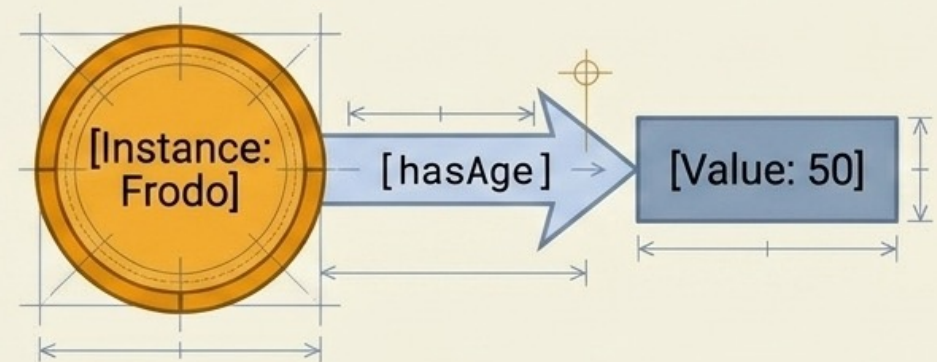
Object Properties

Link an individual to another individual.
(Entity -> Entity)



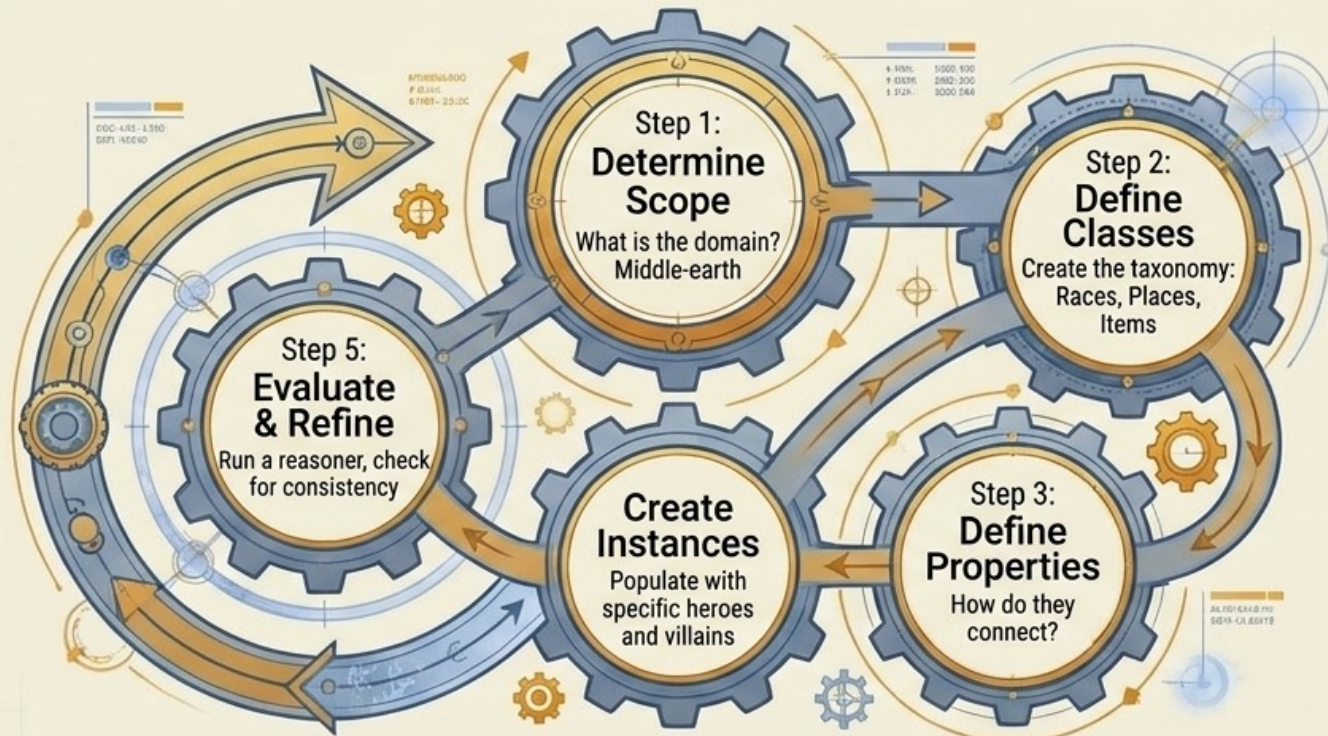
Datatype Properties

Link an individual to an XML Schema
Datatype value or an RDF literal.
(Entity -> String/Integer/Date)



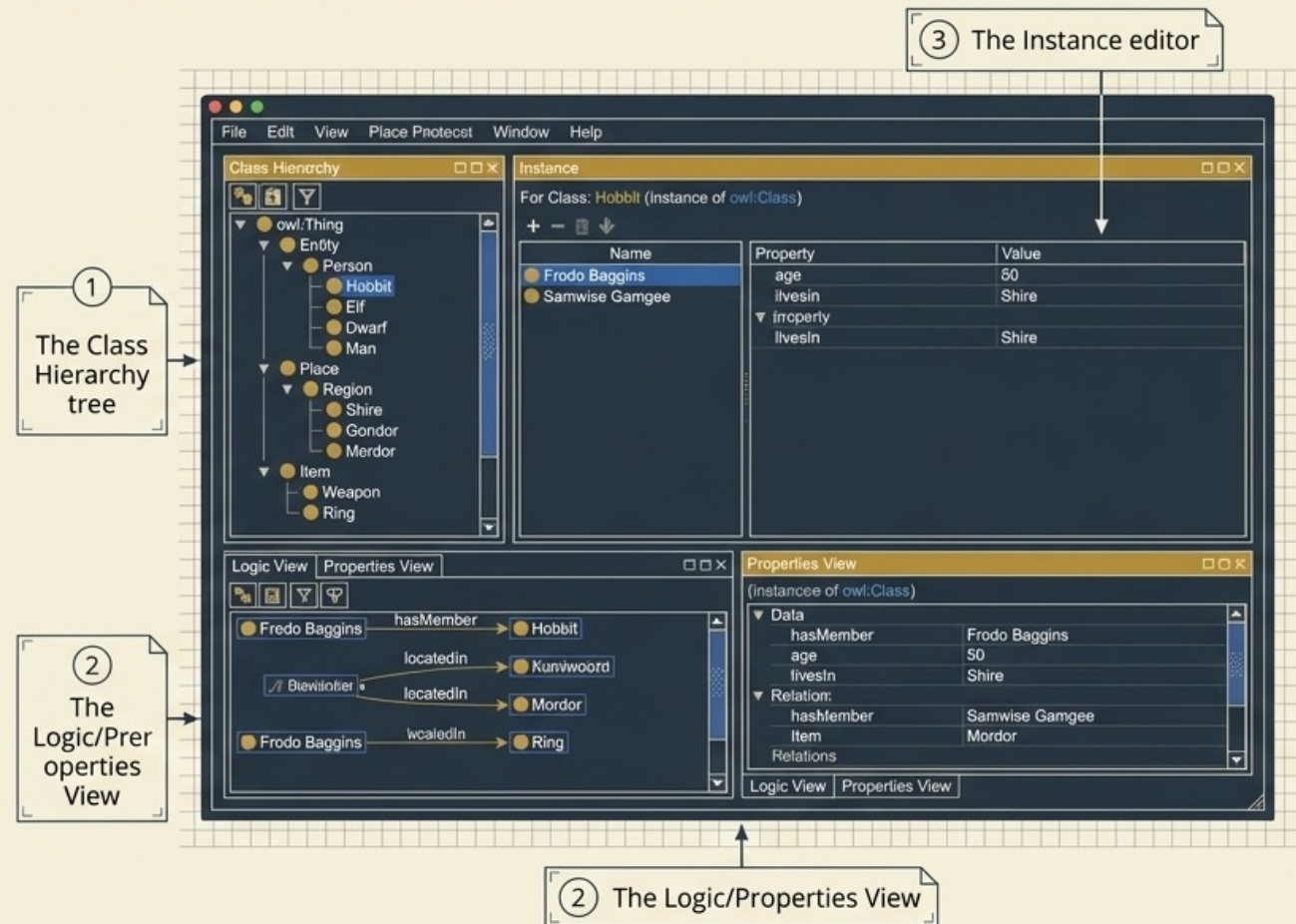
The Ontology Forge: A 5-Step Methodology

Building an ontology is not a linear task. It is a highly iterative process of modeling, testing, and refining as your understanding of the domain evolves.

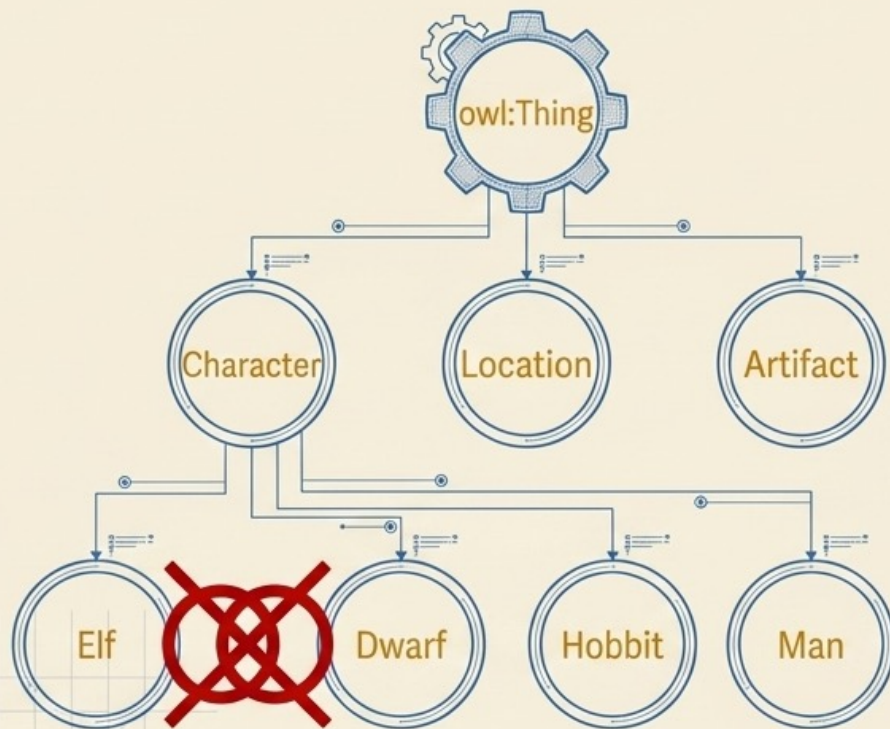


Enter Protégé: The Architect's Workbench

Protégé is the industry-standard, extensible open-source environment for building OWL (Web Ontology Language) ontologies. It provides a visual interface for managing the underlying RDF triples and complex logical axioms without writing raw XML code.



Step 1: Mapping the Taxonomy



'Asserted Hierarchy'

owl:Thing

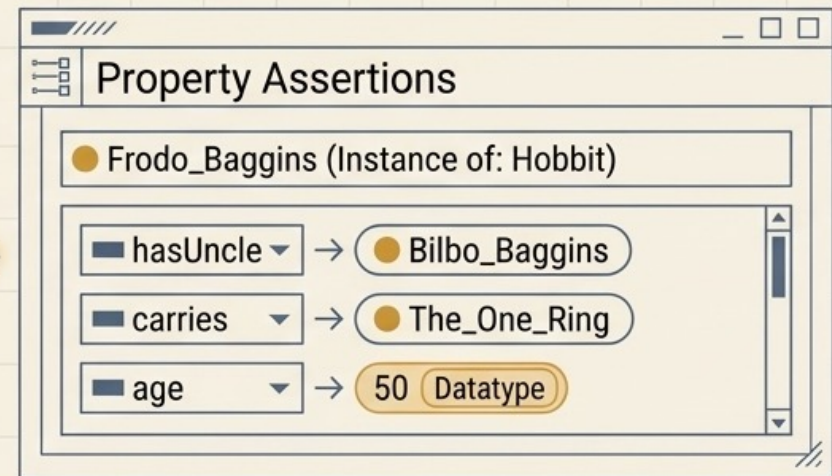
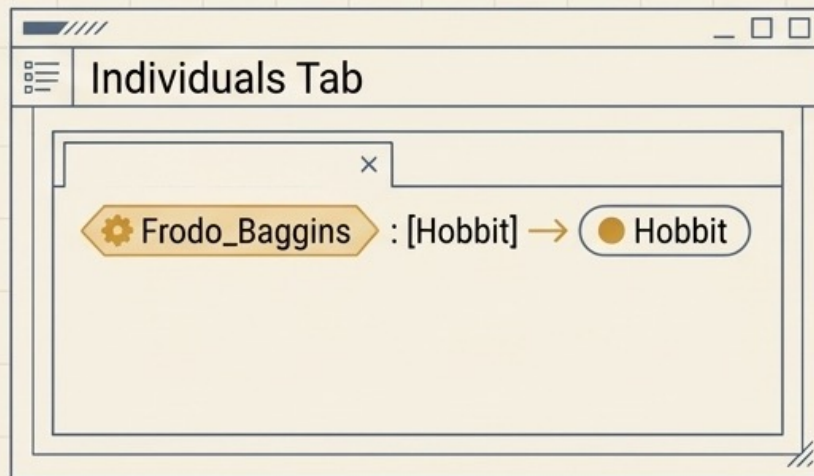
In OWL, every class is implicitly a subclass of the master root class, owl:Thing, which represents the set of all individuals.

Disjoint Classes

An ontology does not assume mutual exclusivity. You must explicitly state that a class cannot overlap. (e.g., An individual cannot be both an Elf and a Dwarf).

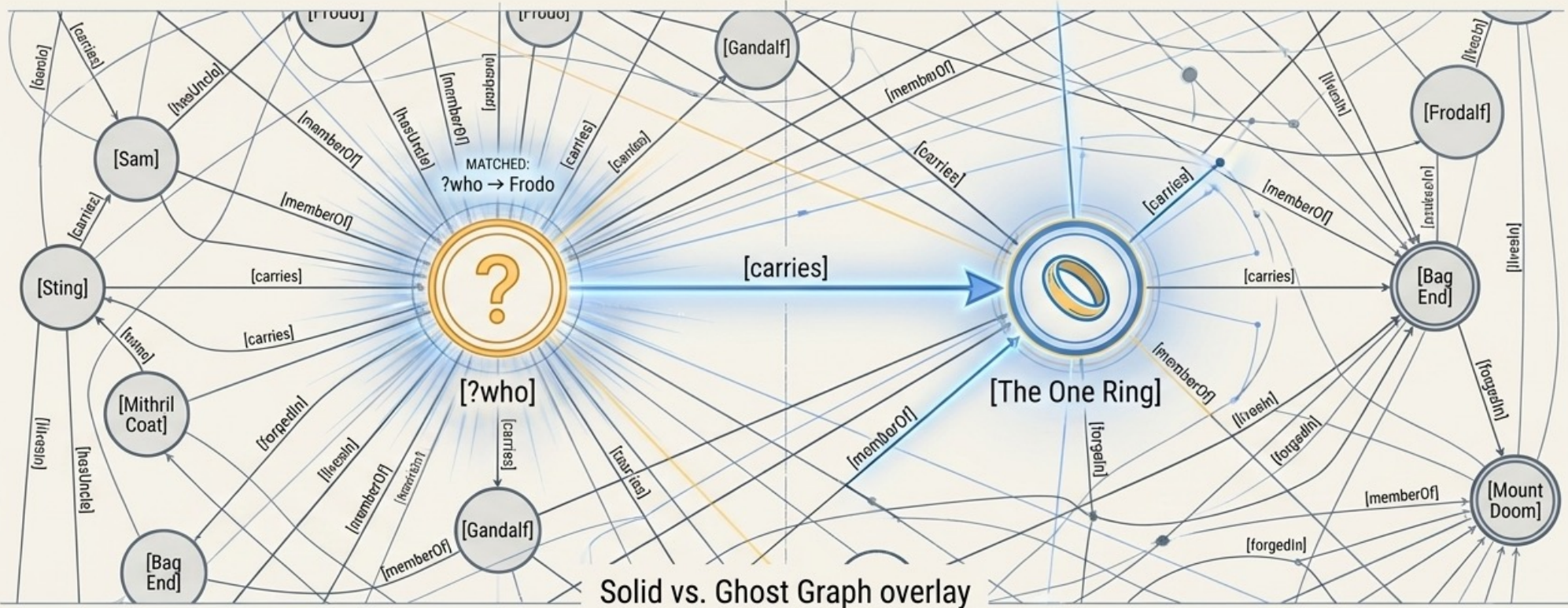
Step 2: Forging Instances and Assertions

Once the taxonomy is built, we populate it with individuals. Because OWL does not use the Unique Name Assumption (UNA), we must uniquely assert properties for each instance to differentiate them, or explicitly declare them as `owl:sameAs` or `owl:differentFrom`.



Querying the Graph: How SPARQL Works

SPARQL does not search by keywords. It searches by **graph pattern matching**. You provide a pattern with “**variables**” (starting with **?**), and the engine finds every subgraph in your ontology that **perfectly fits** that shape.



The Anatomy of a Basic Query

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX lotr: <http://example.org/middle-earth#>

SELECT ?character
WHERE {
    ?character rdf:type lotr:Hobbit .
    ?character lotr:carries lotr:TheOneRing .
}
```

PREFIX: Defines namespaces so we don't have to write full URLs.

SELECT: Identifies the **variables** we want returned in the results.

WHERE: Contains the **basic graph pattern** (the triples) the engine must match.

?character
lotr:Frodo
lotr:Bilbo
lotr:Gollum

Advanced Traversal: Filters and Optional Patterns

FILTER: Applies boolean constraints (math, string matching, inequality) to restrict the matched variables.

```
FILTER (?age > 2000)
```

	Data node	Data node
1	Elrond	Galadriel
2	Elrond	Galadriel

Excludes Legolas

OPTIONAL: Prevents a query from failing if a piece of data is missing, returning results even for incomplete graph patterns.

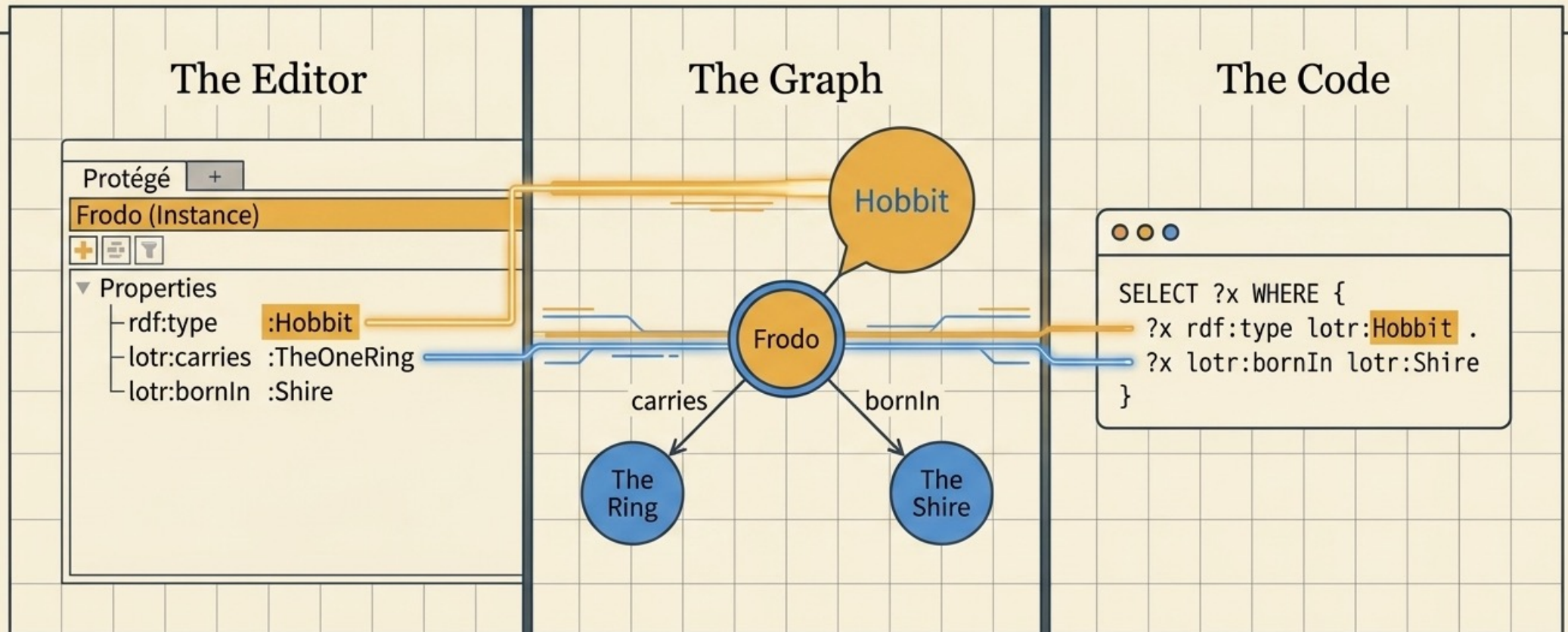
```
OPTIONAL {  
  ?character lotr:hasWeapon ?weapon  
}
```

	Data node	?weapon
1	Aragorn	Andúril
2	Samwise	[Blank]

Returned despite missing data

Three Lenses, One Semantic Reality

The tool, the visual graph, and the query language. Whether you are clicking through Protégé, visualizing nodes, or writing SPARQL, you are manipulating the exact same underlying logic of the Semantic Web.



Your Turn: Architect Your Own Domain

1. **Conceptualize:** Choose a domain of interest (a video game, a historical event, a scientific field, a sport).
2. **Model:** Define at least 10 Classes (with hierarchy), 5 Object Properties, and 3 Datatype Properties.
3. **Forge:** Build the ontology in Protégé and populate it with at least 15 Instances.
4. **Interrogate:** Write 3 distinct SPARQL queries to extract meaningful insights from your knowledge graph. Include at least one FILTER or OPTIONAL clause.



Add Node